Milestone 1 — Formula 1 Podium Prediction

Analytical report

Team 95

Name	ID
Youssef Mohamed Mahmoud Eid	52-2413
Omar Khaled Mohamed Mohamed Ghonima	58-1891
Mohamed Ayman Mohamed Abdelrahman Elsayed	58-22010
Omar Ahmed Mohamed Mohamed Mohamed Elbehery	58-16880

---

1. Introduction

Task. Predict whether a driver finishes on the podium (`positionOrder <= 3`) using only information that is known **after qualifying and before the race starts** (the pre-race cutoff).

Unit of analysis. One driver in one race, i.e. one row per `(raceId, driverId)`. **Target:** `podium` (1 if `positionOrder <= 3`, else 0). The target is imbalanced: about 13% of all entries (1950–2024) and 14% of the modeling rows are podiums.

Dataset. Kaggle *Formula 1 World Championship (1950–2024)*, 14 relational CSV tables (e.g. results 26,759 rows, races 1,125, qualifying 10,494, drivers 861, lap_times 589,081). The raw tables are loaded without modification (with `keep_default_na=False`) and all cleaning is done on copies. The final modeling table has 8,970 rows (2003–2024) and 12 features.

The notebook (`F1_Podium_Prediction.ipynb`) contains all code and plots; this report contains the written answers, the feature justification, the results, the explanations and the limitations.

2. Data audit & cleaning

2.1 What the raw data looks like

`\N` is used as a "missing" placeholder in 6 tables (160,076 sentinel cells in total; e.g. results 122,887, races 11,349, qualifying 11,600, drivers 1,559). Because of it, numeric columns such as `results.position`, `results.milliseconds` or `qualifying.q1` were read as text. Some sentinels are real missing values (`position` is `\N` when the driver did not finish), others mean "did not happen" (`q3` is empty for drivers knocked out in Q1/Q2). Durations (`1:27.452`) and dates are stored as strings. 91 extra rows share the same `(raceId, driverId)`, almost all in the 1950s. The target is imbalanced (~13% podiums) and the podium rate drops quickly with grid position.

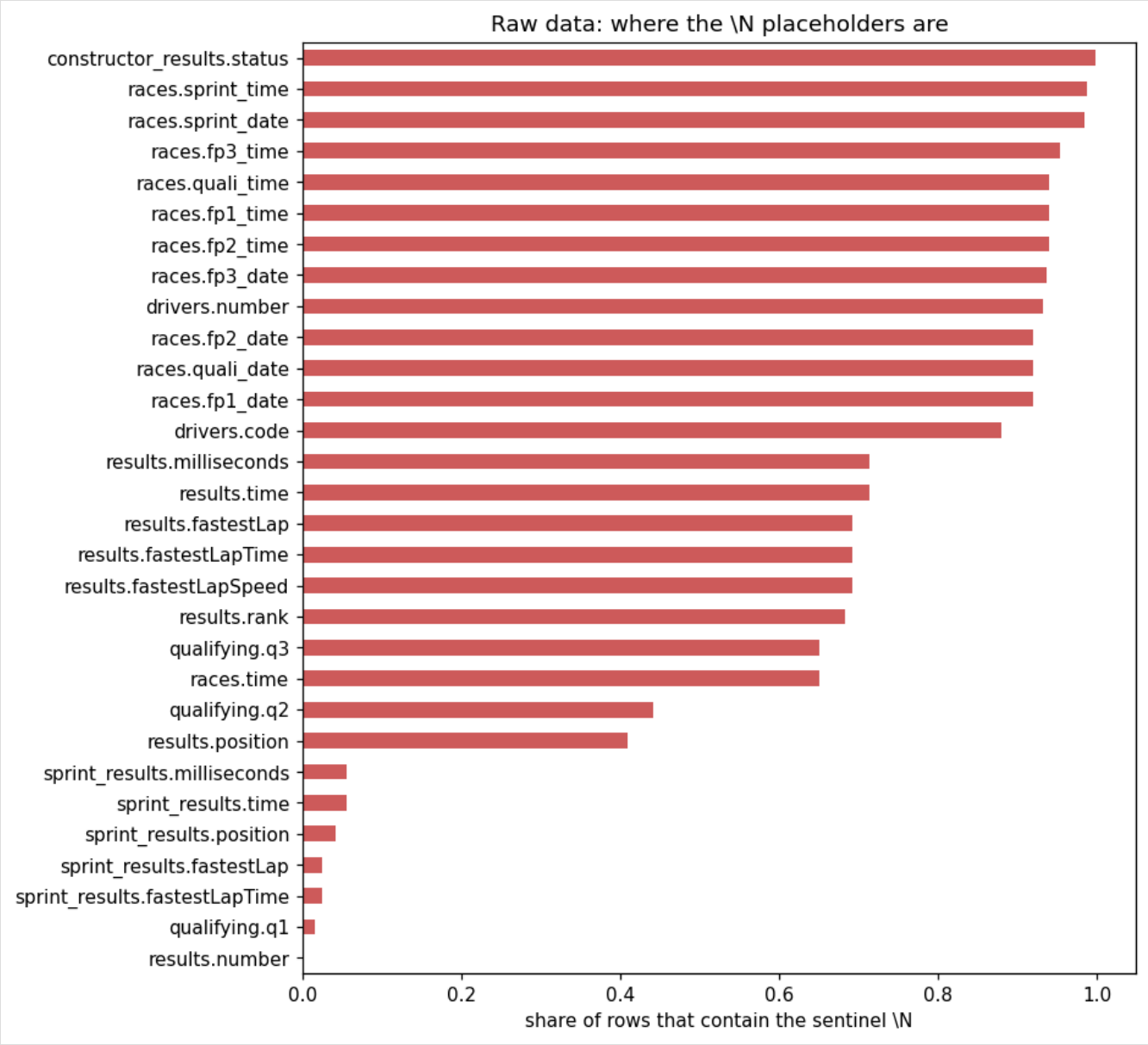


Figure 1. Raw data: \N sentinel cells per table / column.

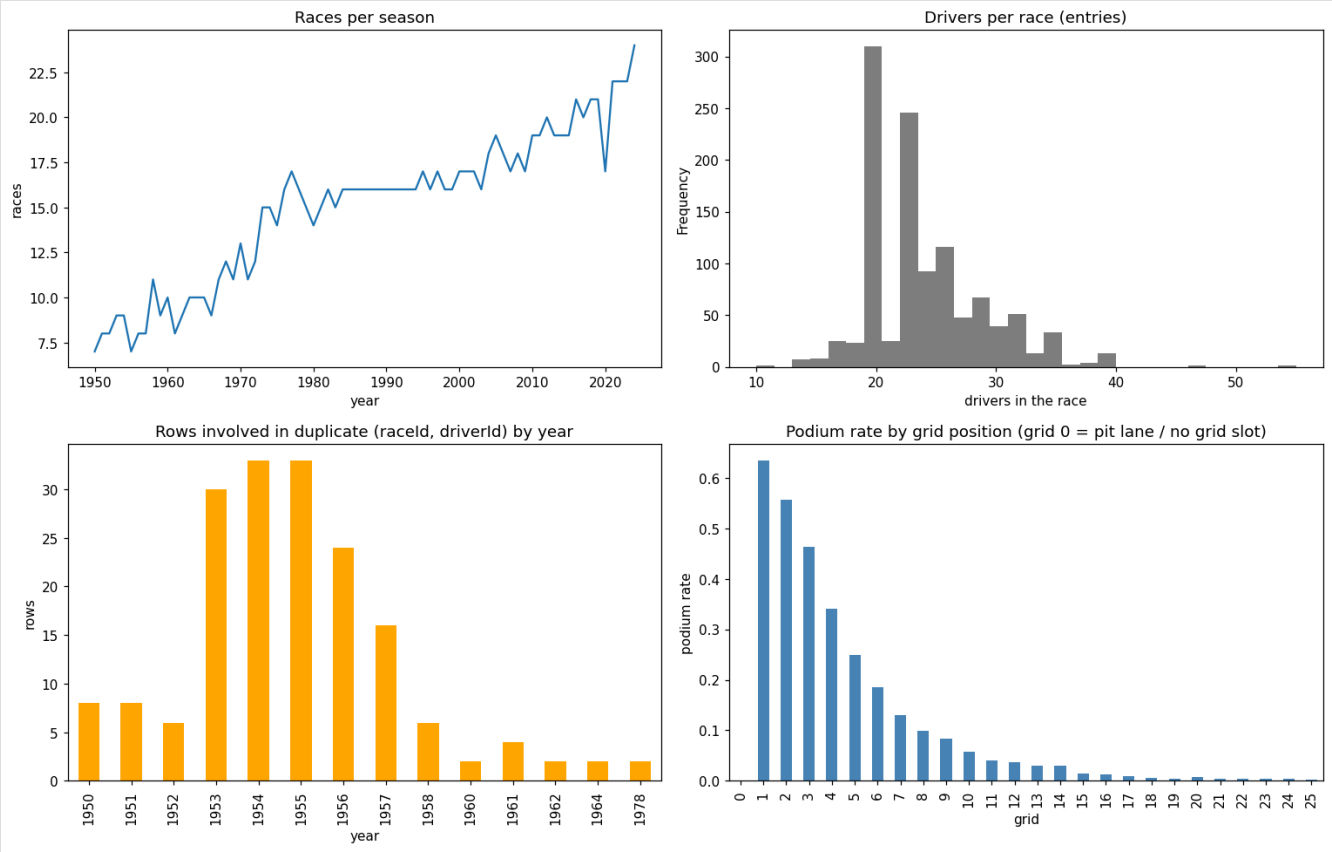


Figure 2. Raw data overview (EDA before cleaning).

2.2 Sentinel handling and semantic typing

The rules are applied automatically to every table, not by hand:

- Replace `\N` (and empty strings) with a real missing value NaN.
- Strip extra spaces from text values.
- If every non-missing value of a text column is a number, convert the column to numeric.
- Date columns (`date`, `*_date`, `dob`) become datetime.
- Duration strings (`m:ss.sss`, `h:mm:ss.sss`, `ss.sss`) become numeric seconds (new `*_s` columns).
- Fix label inconsistencies: circuit country *United States* vs *USA*, and driver nationality *Argentinian* vs *Argentine*.

Afterwards no `\N` cells remain. 517 pit-stop durations are longer than 60 s (red-flag stops); they are kept but noted.

2.3 Primary keys and foreign keys

A primary key must be unique and non-null; a foreign key must exist in its parent table (no orphans). We checked 14 primary keys (single and composite, e.g. `raceId + driverId + lap` for lap_times) and 22 foreign keys. **Result: all primary keys are unique and non-null (0 null, 0 duplicate keys) and all foreign keys reconcile (0 null values, 0 orphans).**

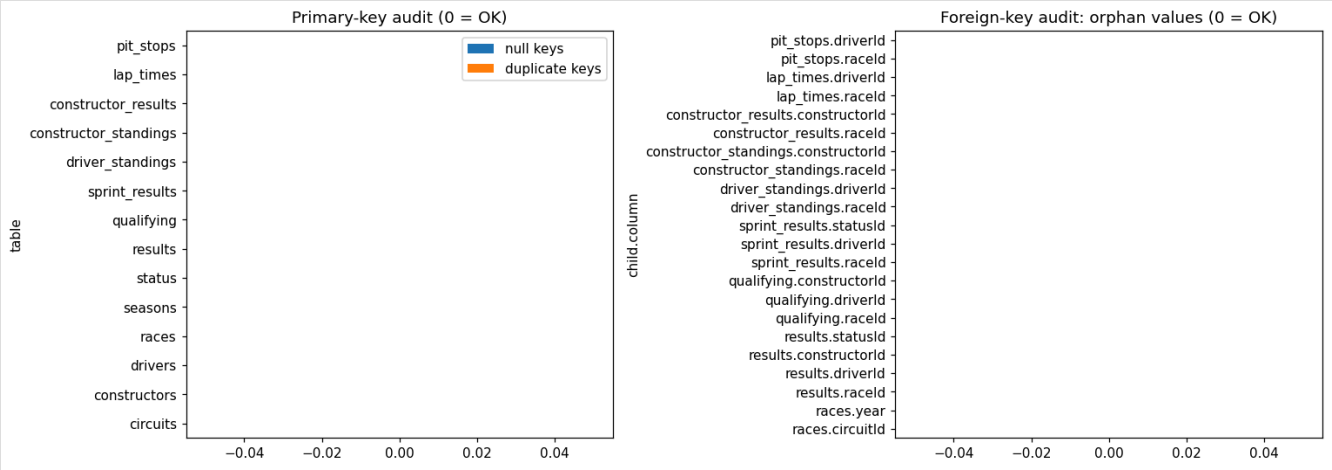


Figure 3. Key audit: primary-key and foreign-key checks.

2.4 Grain and the duplicate-driver rule

`resultId` is unique, but a driver can have two result rows in the same race: in the 1950s drivers shared cars, so the same driver appears with two cars. There are 176 rows involved, 91 extra rows to remove, in the years 1950–1962, 1964 and 1978.

Rule: keep the row with the best (lowest) `positionOrder` for that driver in that race.

Why: the question is "did this driver finish on the podium in this race?". If any of the cars the driver drove reached the podium, the official result credits the driver with that podium (points were shared), so the best row is the correct answer for the target. It is also simple and deterministic. These duplicates are all before 1979, so they do not touch the 2003–2024 modeling data, but the rule is needed for the data-engineering questions that use all years. After the rule, results go from 26,759 to 26,668 rows and `(raceId, driverId)` is unique.

2.5 Impact of the cleaning

Table 1. Impact of the cleaning.

	before cleaning	after cleaning
\N sentinel cells	160,076	0
text (object) columns	57	40
results rows (driver-race)	26,759	26,668
distinct circuit countries	35	34
distinct nationalities	43	42

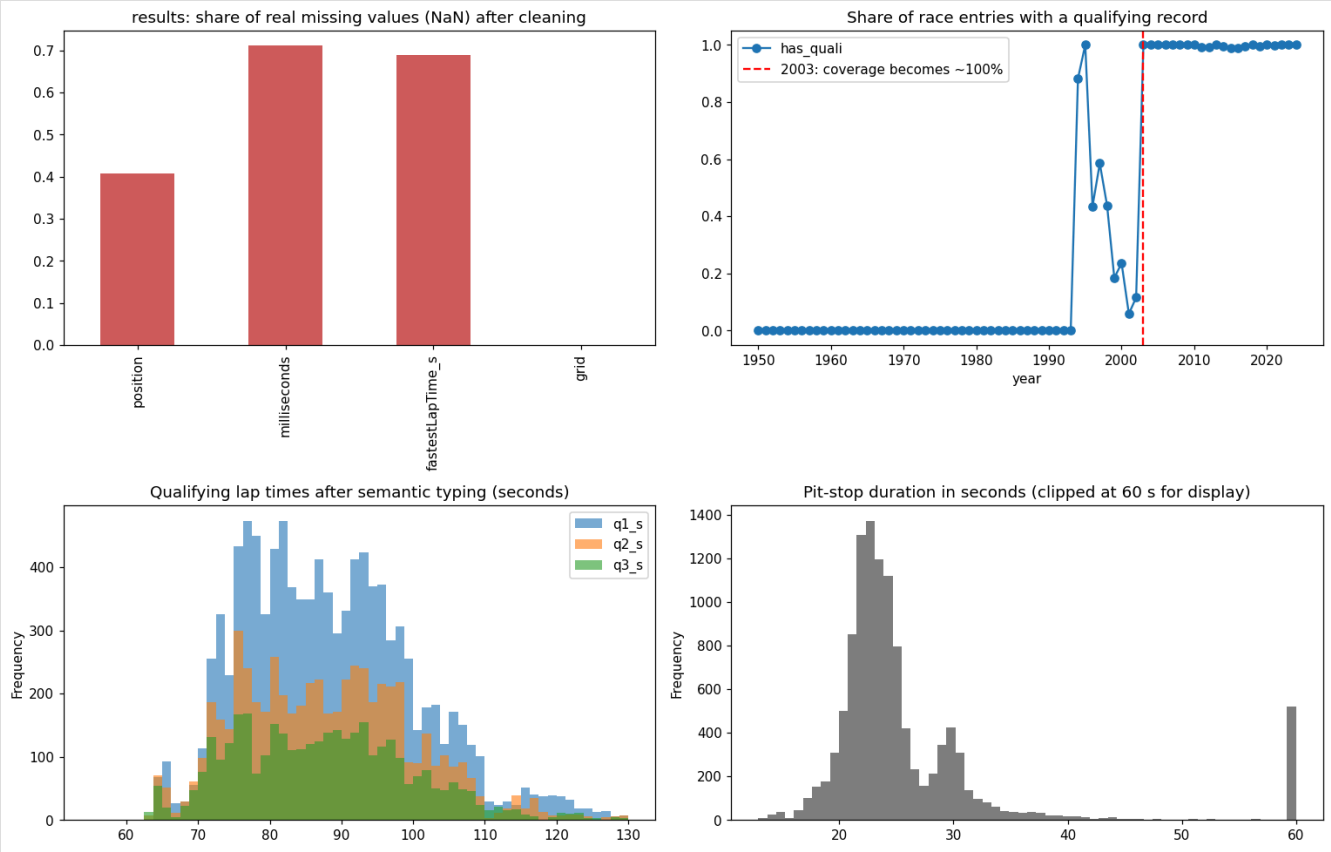


Figure 4. EDA after cleaning.

- All sentinels are gone; numeric columns are real numbers and the remaining missing values are true NaNs that can be counted and handled.
- Durations are numbers in seconds (qualifying laps ~60–130 s, pit stops ~20–30 s, a few red-flag stops > 60 s) and dates are real dates.
- Duplicate driver-race rows were removed and two label inconsistencies were fixed.
- **Qualifying coverage drift:** the dataset only has qualifying records from 1994, and between 1996 and 2002 most are missing. From 2003 coverage is ~100%. Because qualifying is one of our feature groups, we model on **2003–2024** (the full history is still used to compute past form).

3. Data-engineering questions

All three need multi-table joins (`results` + `races` + `circuits` + `qualifying` / `drivers` / `status` / `constructors`).

Q1. Which circuits convert a front-row start into a podium most often, and does that change after the 2014 hybrid era?

Method.

- **Join path:** `results` → `races` (year, circuit) → `circuits` (name) + `qualifying` (qualifying position).
- **Grain:** one row per driver-race; a front-row start is `grid` 1 or 2.
- **Qualifying vs grid:** qualifying position is where the driver qualified; grid is where they actually started (penalties can move them back). We compute the conversion using both.
- **Missingness / window:** qualifying records are only complete from 2003, so we compare 2003–2013 (pre-hybrid) with 2014–2024 (hybrid). Only circuits with at least 8 front-row starts (about 4 races) in both eras are ranked.
- **Pre-race cutoff:** grid and qualifying are both known before the race; the podium is the outcome.

Answer.

- **Most frequent converters.** 16 circuits have at least 8 front-row starts in both eras. Pre-hybrid (2003–2013): Monza and Monaco (77%), then Silverstone (73%), Spa and Suzuka (72%). Hybrid era (2014–2024): Suzuka (89%), Yas Marina (86%), Monaco (85%), then Barcelona and Spa (82%).
- **Does the ranking change? It is unstable.** The Spearman rank correlation between the two eras is only about 0.21. Yas Marina moved from 12th to 2nd, Monza fell from 1st to 11th and Hockenheim from 6th to 15th. However, the rate of each circuit rests on only 8–22 starts, so much of this reshuffling can be small-sample noise. **The robust finding is that front-row conversion went up overall** in the hybrid era, from 68% to 76% (by grid). This fits the hybrid era being dominated by one or two teams (Mercedes 2014–2020, Red Bull 2022–2024), whose cars rarely drop out of the top 3.
- **Qualifying vs grid.** Using qualifying position gives slightly lower conversion (67% / 74%). 38 drivers qualified on the front row but did not start there (grid penalties); only 29% of them reached the podium. So qualifying and grid are different information, both available before the race; we use both as features plus their difference (`grid_penalty`).
- **Caveat.** Each circuit has only 8–22 front-row starts per era, so differences of about 10 percentage points between single circuits can be noise.

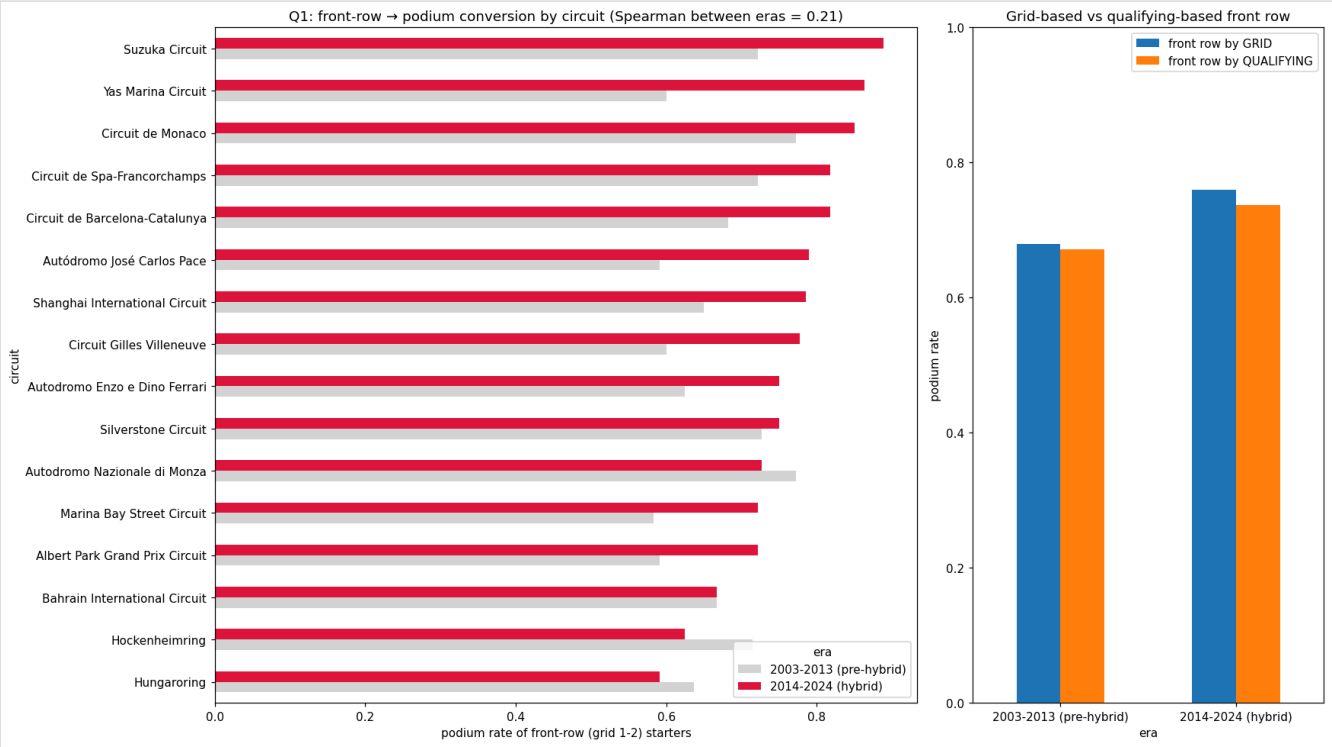


Figure 5. Q1: front-row to podium conversion by circuit and era.

Table 2. Selected rows of the Q1 conversion table (grid-based; the notebook lists all 16 circuits).

circuit	2003–2013	2014–2024	rank pre	rank hybrid
Suzuka Circuit	0.722	0.889	4	1
Yas Marina Circuit	0.600	0.864	12	2
Circuit de Monaco	0.773	0.850	1	3
Circuit de Barcelona-Catalunya	0.682	0.818	7	4
Circuit de Spa-Francorchamps	0.722	0.818	4	4
Autodromo Nazionale di Monza	0.773	0.727	1	11
Hockenheimring	0.714	0.625	6	15
Hungaroring	0.636	0.591	10	16

Q2. Do drivers podium more often at a home race, after controlling for starting position?

Method.

- **Join path:** `results` → `races` → `circuits` (country) and `results` → `drivers` (nationality). A small dictionary maps nationality to country (e.g. British → UK).
- **Home race:** driver country equals circuit country. Only drivers whose country has hosted at least one Grand Prix are used (14 nationalities without a home circuit are excluded).
- **Controlling for starting position:** we split into grid bins (P1, P2–3, P4–6, P7–10, P11+), compare home vs away within each bin, then average the within-bin differences weighted by home starts, giving an "adjusted" difference.
- **Missingness / cutoff:** rows with `grid = 0` (pit lane / no grid slot) are excluded because the starting position is unknown. Grid is pre-race information.

Table 3. Podium rate, home vs away, within grid bins.

grid bin	away podium rate	home podium rate	away starts	home starts
P1	0.654	0.576	941	92
P2-3	0.514	0.535	1816	172
P4-6	0.260	0.245	2706	261
P7-10	0.090	0.088	3689	387
P11+	0.016	0.016	11219	1411

Answer.

- **Raw comparison:** over 1950–2024 drivers at home podium slightly less often (**−2.1 points**) because home drivers start further back (mean grid 13.4 vs 11.8); many local drivers only entered their home Grand Prix, often in weaker cars.
- **Controlling for starting position:** inside each grid bin the rates are almost identical (P2–3: 53.5% vs 51.4%; P4–6: 24.5% vs 26.0%; P7–10: 8.8% vs 9.0%; P11+: 1.6% vs 1.6%). The only visible gap is from pole (P1), where home drivers did *worse* (57.6% vs 65.4%), but that bin has only 92 home starts, so it is a small sample. The home groups per bin have 92–1,411 starts, against 941–11,219 away. The grid-adjusted difference is about **−0.3 points**. For 2003–2024 only (364 home starts) it is −1.9 points.
- **No.** Once starting position is accounted for there is no evidence of a home advantage, so we did not add a "home race" feature.

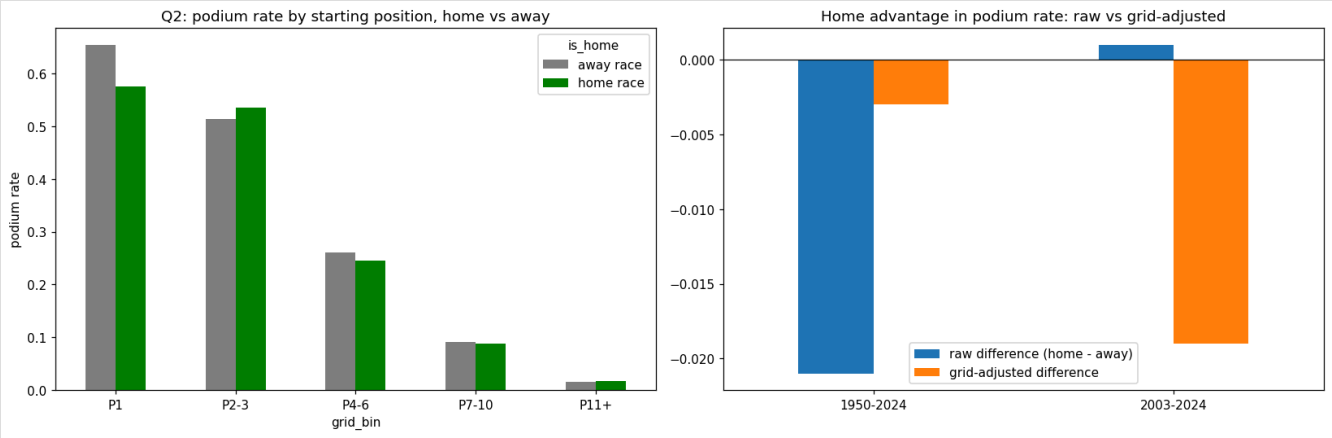


Figure 6. Q2: home vs away podium rate, raw and grid-adjusted.

Q3. Which constructors have the highest mechanical-retirement rate in 2014–2021 vs 2022–2024?

Method.

- **Join path:** `results` → `status` (why the race ended) → `constructors` (team) → `races` (year).
- **Mechanical retirement:** the status names a car component failure (engine, gearbox, hydraulics, brakes, power unit, ERS, ...). Excluded: crashes/driver errors (Accident, Collision, Spun off, Damage, Debris), regulation decisions (Disqualified, Excluded, Underweight, 107% Rule) and unclear reasons (Retired, Withdrew, Illness, Out of fuel, tyres/punctures, wheel nut).
- **Grain and rate:** one row per car start; rate = mechanical retirements / race starts (starts exclude Withdrew, Did not qualify, Did not prequalify, 107% Rule). Only constructors with at least 30 starts in an era are ranked.
- **Cutoff:** this is retrospective (status is post-race), so the pre-race cutoff does not apply; it is **not** used as a model feature.

Table 4. Top 5 mechanical-retirement rates per era.

2014–2021	starts	mech.	rate	2022–2024	starts	mech.	rate
Caterham	34	8	0.235	Alfa Romeo	88	11	0.125
Lotus F1	75	13	0.173	Alpine F1 Team	136	15	0.110
Toro Rosso	242	39	0.161	AlphaTauri	88	7	0.080
McLaren	318	44	0.138	Ferrari	136	9	0.066
Renault	200	27	0.135	Williams	134	8	0.060

Answer.

- **2014–2021:** Caterham 23.5% (only 34 starts, 2014), Lotus F1 17.3%, Toro Rosso 16.1%, McLaren 13.8% (the McLaren-Honda years 2015–2017) and Renault 13.5%.
- **2022–2024:** Alfa Romeo 12.5%, Alpine 11.0%, AlphaTauri 8.0%, Ferrari 6.6% and Williams 6.0%.
- **Trend:** overall reliability improved. The all-team rate per season is clearly lower in 2022–2024 than in 2014–2015, so even the worst teams of 2022–2024 are below the worst of 2014–2021.
- **Check on the excluded generic "Retired" label:** it covers only about 1% of starts in both periods (1.2% in 2014–2021 vs 0.9% in 2022–2024), so excluding it does not create the trend.
- **Caveats:** renamed teams (Toro Rosso → AlphaTauri → RB, Renault → Alpine, Sauber → Alfa Romeo) have different `constructorId`s and are counted separately; status labels are coarse (a generic "Retired" may hide component failures and was excluded on purpose); 2022–2024 has only 3 seasons.

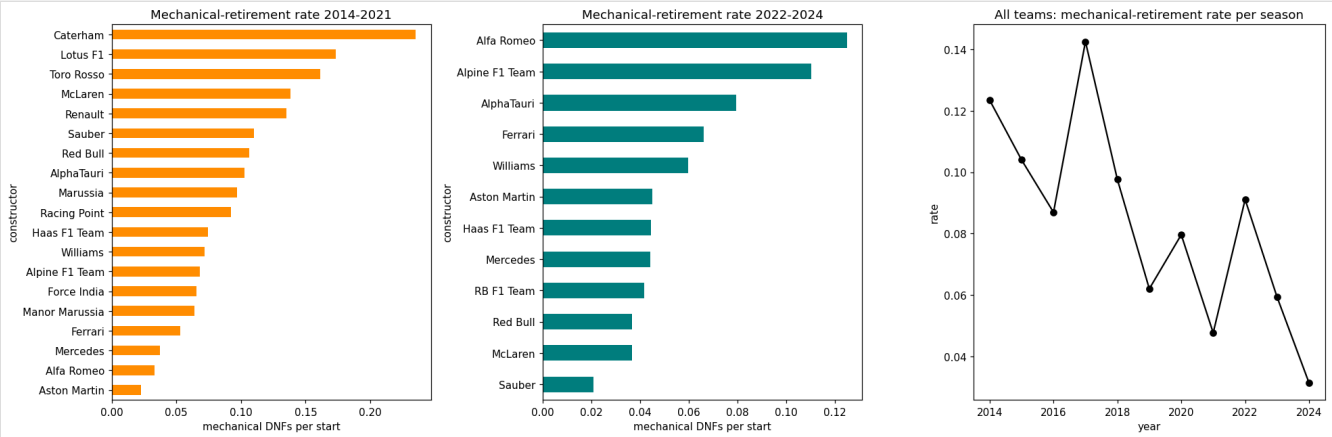


Figure 7. Q3: mechanical-retirement rate by constructor and per season.

4. Features

Prediction cutoff: after qualifying, before the race. A feature is allowed only if it is known at that moment. The 12 features are in 4 groups, chosen from the EDA hypothesis that starting position plus current car/driver strength decide podiums.

Table 5. The 12 selected features.

Group	Feature	Meaning	Why valid at the pre-race cutoff
grid	`grid`	starting position (pit-lane start = back of grid)	Set by qualifying and penalties, published before the start.
grid	`grid_penalty`	`grid - quali_position` (positive = moved back)	Difference of two pre-race quantities.
qualifying	`quali_position`	qualifying classification	Result of the qualifying session of this race, which ends before the race.
qualifying	`quali_gap_pct`	% slower than pole (best of Q1/Q2/Q3), pure pace	Computed from this race's qualifying times only.
form	`drv_podium_rate_last5`	share of podiums in the driver's previous 5 races	Rolling window over races k-5 ... k-1 only (checked by hand in the notebook).
form	`drv_avg_finish_last5`	average finishing position in the previous 5 races	Same: previous races only.
form	`con_podium_rate_last5`	share of the team's previous 5 races with a podium	Same: previous races only.
form	`drv_races_before`	experience: number of previous starts	Count of earlier races, excludes the current one.
standings	`drv_standing_before`	driver championship position before this race	Taken after the previous round, so it does not include this race's points.
standings	`drv_points_share`	driver points / leader's points before this race	Same; a share is used because the points system changed (10-8-6 to 25-18-15).
standings	`con_standing_before`	constructor championship position before this race	Same as driver standing.
standings	`con_points_share`	constructor points / leader's points before this race	Same as driver points share.

Fields excluded because of leakage (known only after the race or including the current race):

- `positionOrder`, `position`, `points`, `laps`, `time`, `milliseconds` of the current race
- `statusId` (finished / retired) of the current race
- fastest lap / rank of the current race, pit stops and lap times of the current race
- `driver_standings` / `constructor_standings` of the current `raceId` (they include this race's points); we use the standings of the previous round instead

Leakage check. For Lewis Hamilton we recomputed the form feature by hand: the value used for race *k* equals the mean of races *k-5* ... *k-1* only. The check passed. The table has exactly one row per `(raceId, driverId)`.

Evidence for the chosen features

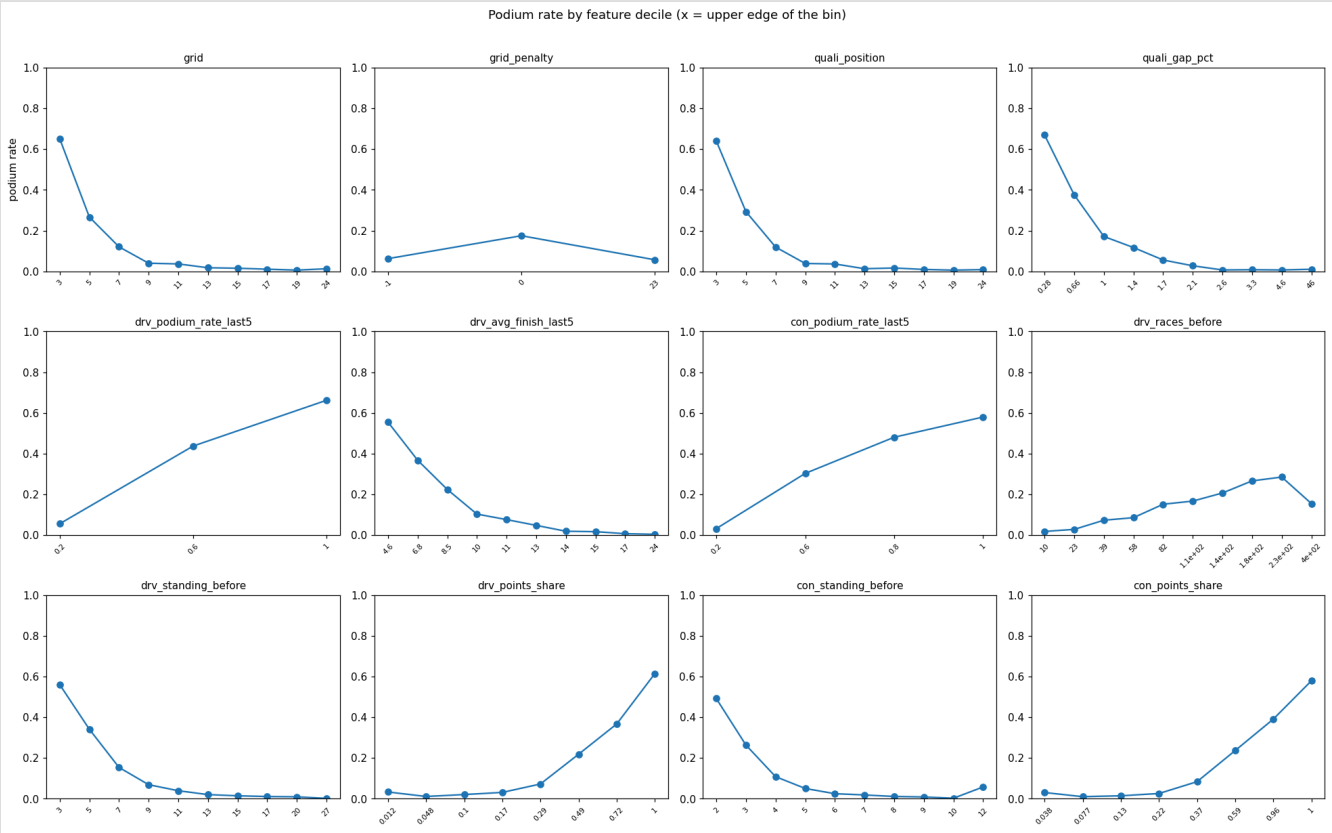


Figure 8. Podium rate across the deciles of each feature.

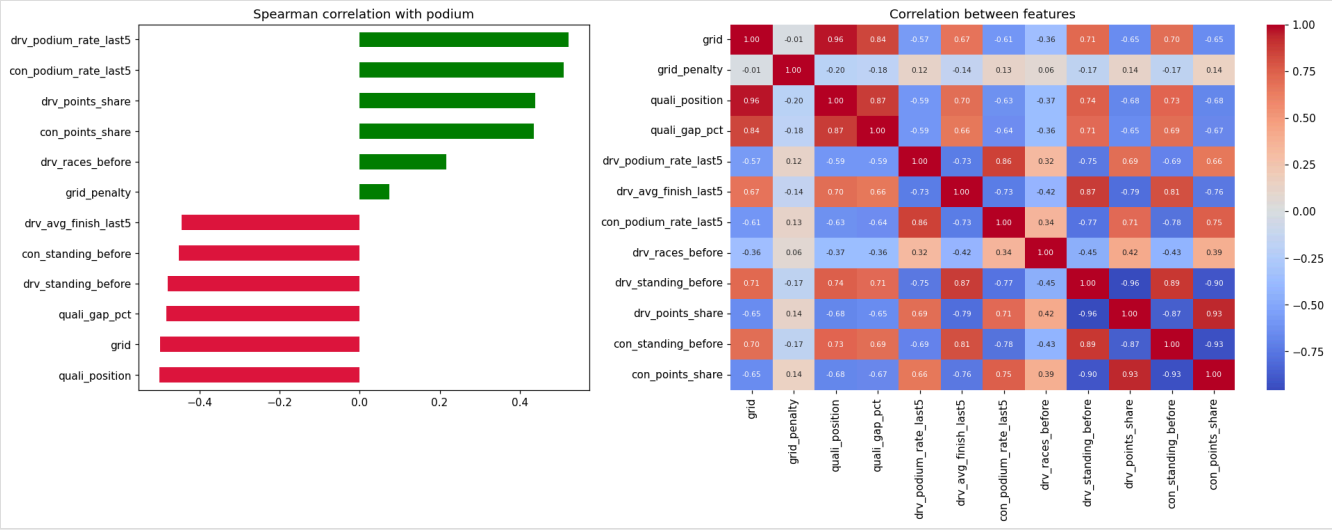


Figure 9. Correlation with the target and between features.

Table 6. Correlation with the target and missing values per feature.

Feature	Spearman with podium	Missing share
quali_position	−0.501	0.003
grid	−0.497	0.000
quali_gap_pct	−0.482	0.016
drv_standing_before	−0.479	0.068
con_standing_before	−0.451	0.055
drv_avg_finish_last5	−0.445	0.010
grid_penalty	0.074	0.003
drv_races_before	0.217	0.000
con_points_share	0.434	0.000
drv_points_share	0.438	0.000
con_podium_rate_last5	0.509	0.005
drv_podium_rate_last5	0.522	0.010

- **Grid and qualifying:** the strongest signals (about −0.50). The podium rate falls from ~75% for the front positions to almost 0 after P8. `quali_gap_pct` adds pace (P1 and P2 can be 0.02% or 1% apart). `grid_penalty` is almost always 0 (correlation 0.07) but Q1 showed that drivers moved back by penalties podium much less often, so we keep it.
- **Form:** `drv_podium_rate_last5` (+0.52) and `con_podium_rate_last5` (+0.51) are among the strongest single features; `drv_races_before` (+0.22) is weaker.
- **Standings:** standing and points share (±0.44–0.48) describe the same "car strength" idea over the whole season so far.
- **Redundancy:** grid and quali_position are almost the same (0.9+), and form and standings are strongly correlated. All of them measure "how fast is this car/driver"; the ablation (Section 7) checks whether each group still adds something.
- **Missing values** are small: standings 5–7% (first race of a season, new drivers), form about 1% (debutants), qualifying below 2%. They are handled by median imputation inside the pipeline.

5. Split & preprocessing

Time-based split by season, not random. The real use case is predicting future races from past ones; a random split would put races of the same weekend/season in both train and test and over-estimate performance. The form and standings features are built from the past, so a time split keeps everything chronological. Validation is used for **every choice** (hyperparameters, probability threshold, final model). The test seasons are only **reported** (model comparison, ablation, XAI); they are never used to choose anything. The test set is also a new regulation era (2022 cars), a realistic and harder test.

Table 7. Time split.

	seasons	rows	races	podium rate
train	2003–2017	5,991	279	0.14
validation	2018–2021	1,620	81	0.15
test	2022–2024	1,359	68	0.15

Preprocessing (fitted on the training set only, then applied to validation and test): (1) missing values are replaced by the median (`SimpleImputer`), which is robust to the outliers in `quali_gap_pct`; (2) standardisation (`StandardScaler`) for Logistic Regression and the neural network, which are sensitive to

scale. The Random Forest only gets the imputer, because trees split on thresholds and scaling does not change them.

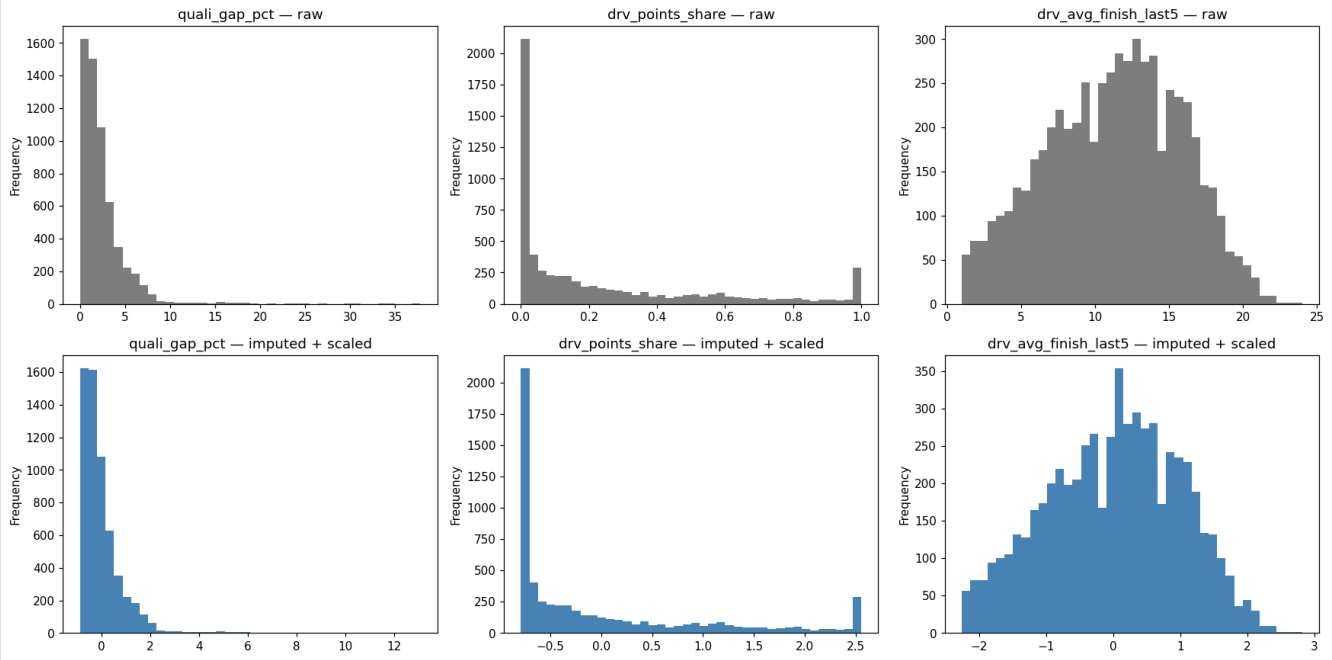


Figure 10. Effect of imputation and scaling on the data.

Effect. After imputation there are no missing values (e.g. `drv_standing_before` had 454 missing in training); the imputed rows pile up at the median (visible spike). After scaling every feature has mean 0 and standard deviation 1, so no feature dominates just because of its units (`drv_races_before` goes up to ~300 while shares are between 0 and 1). The shape of each distribution does not change.

Preprocessing-order experiments

The model is fixed (Logistic Regression baseline) and only the order of the steps changes.

Experiment A: fit the imputer/scaler before vs after the split. Wrong order: median and mean/std on the whole 2003–2024 table, then split. Correct order: split first, fit on training only.

Table 8. Experiment A.

	val PR-AUC	test PR-AUC	val ROC-AUC	test ROC-AUC
fit before split (all data)	0.6976	0.7001	0.9309	0.9349
fit after split (train only)	0.6976	0.7002	0.9309	0.9349

Almost no difference. The training medians and the all-data medians are very similar (only `drv_races_before`, 68 vs 82, and `quali_gap_pct` changed noticeably; the points-share medians moved by at most 0.002), few values are imputed, and the logistic regression barely reacts to small scaling shifts. **We still use the correct order** (train only), because the leaky order uses statistics of the validation and test seasons, which are not available in real use. In this dataset the leak is simply small.

Experiment B: compute the form features before vs after filtering to 2003+. Attempt 1 filters to 2003+ first and then computes the rolling form (cold start). Attempt 2 (our pipeline) computes form on the full 1950–2024 history first, then filters. Both are leakage-safe; the question is whether the order loses information.

Table 9. Experiment B.

order	model	val PR-AUC	val ROC-AUC
filter → form (cold start)	LogReg	0.6988	0.9308
filter → form (cold start)	RandomForest	0.7191	0.9418
form on full history → filter	LogReg	0.6976	0.9309
form on full history → filter	RandomForest	0.7219	0.9418

Computing form after the filter gives a cold start: more missing form values in training (246 vs 174) and in 2003 the average driver appears to have only 7 previous races instead of 80, which is wrong. Performance is about the same (LR 0.699 cold start vs 0.698; RF 0.719 cold start vs 0.722 full history). **We keep "full history first"** because it produces the true values (real experience and form in 2003). It is also what the inference function sees in real use, where the full history is always available. **Lesson:** see the summary after Experiment C.

Experiment C: imputation before vs after scaling. Attempt 1 (our pipeline): median-impute first, then standardise. Attempt 2: standardise first (the scaler ignores NaNs), then median-impute the scaled values.

Table 9b. Experiment C.

order	val PR-AUC	test PR-AUC	train std of drv_standing_before after prep
impute → scale	0.6976	0.7002	1.0000
scale → impute	0.6977	0.7000	0.9615

The results are practically identical. `StandardScaler` ignores missing values when it computes the mean/std, so the two orders only differ in whether the imputed medians are included in the std (1.0000 vs 0.9615 for `drv_standing_before`), a tiny change with so few missing values. **Lesson (A, B and C):** the differences are at most about 0.003 PR-AUC, which is smaller than the seed-to-seed variation measured in Section 6 (RF std about 0.004). We choose the order for **validity** (no information from validation/test seasons, true history values), not because it scored better.

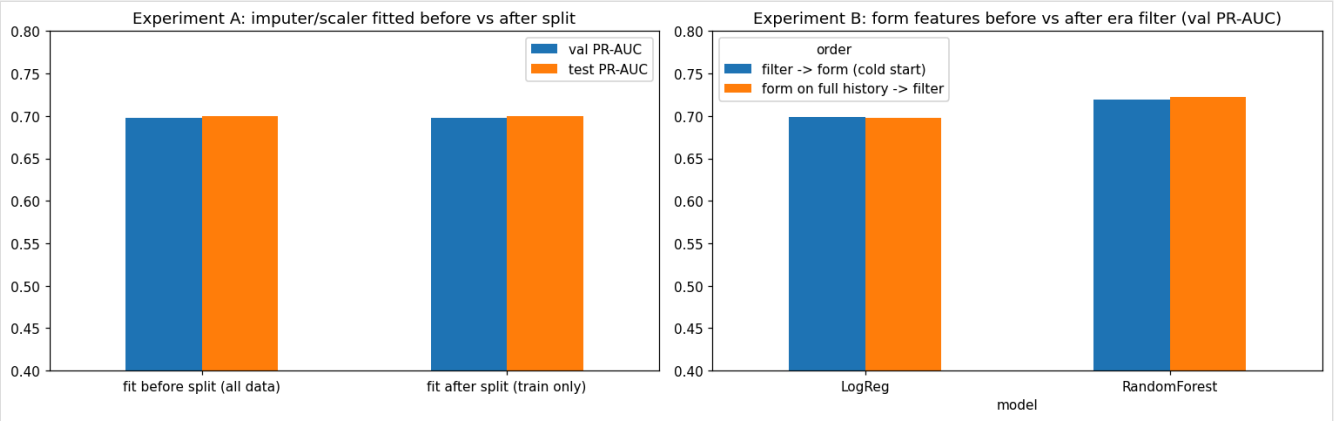


Figure 11. Results of the preprocessing-order experiments.

6. Models & results

All models use the same rows (train/val/test) and the same 12 features, so scores are directly comparable.

#	Attempt	Role
0	Rule "the better the grid slot, the higher the podium chance"	sanity reference (no learning)
1	Logistic Regression	baseline statistical model
2	Random Forest (6 hyperparameter variants)	2nd statistical model
3	Shallow neural network (2 architectures)	neural model

How they work and their limitations.

- **Logistic Regression:** a weighted sum of the scaled features passed through a sigmoid gives a probability; linear decision boundary, easy to interpret. *Limitation:* cannot learn interactions or non-linear effects unless we create them (e.g. grid matters differently for strong vs weak teams). We tried C in {0.01, 0.1, 1, 10}; best on validation C = 1 (val PR-AUC 0.6976).
- **Random Forest:** many decision trees, each trained on a bootstrap sample with random feature subsets; the probability is the average vote. Captures non-linear effects and interactions. *Limitations:* less transparent, probabilities can be poorly calibrated, may overfit if trees are very deep.
- **Shallow NN:** 1–2 hidden layers of ReLU neurons and a sigmoid output, trained with Adam on binary cross-entropy, early stopping on validation loss (patience 10). NN-A: 1 hidden layer (32); NN-B: 2 hidden layers (64-32) with Dropout 0.2. *Limitations:* needs scaling, results vary with the random seed, hardest to explain, and with ~6k training rows it has no real advantage over trees.

Hyperparameter variants. Random Forest variants (selected on validation): max_depth 4 with min_samples_leaf 10 gives the best val PR-AUC (0.7219). Deep trees overfit: depth 12 with leaf 1 has train PR-AUC 0.9924 but validation 0.6914. Neural networks: NN-A (32) val PR-AUC 0.7053 (73 epochs); NN-B (64-32, dropout) 0.7277 (44 epochs) and was selected.

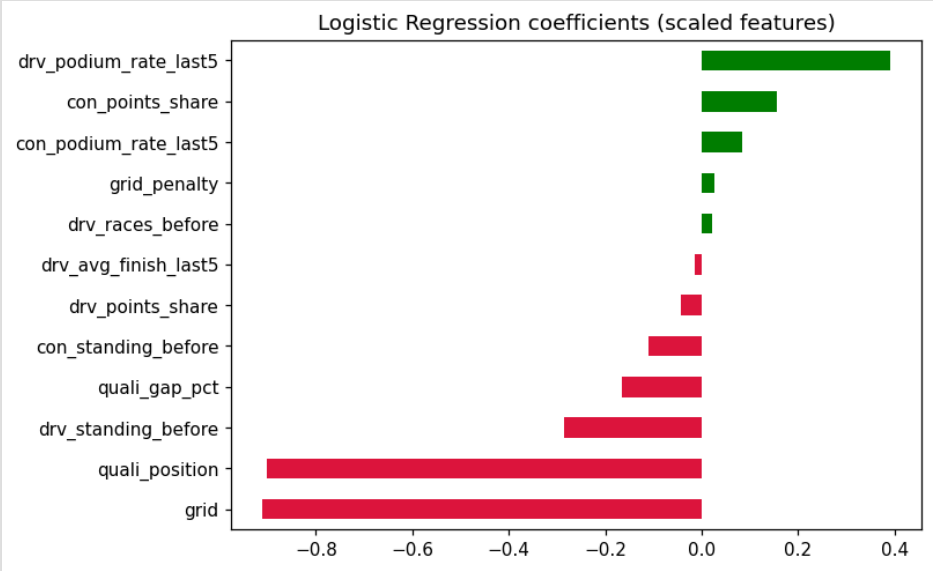


Figure 12. Logistic Regression coefficients (baseline).

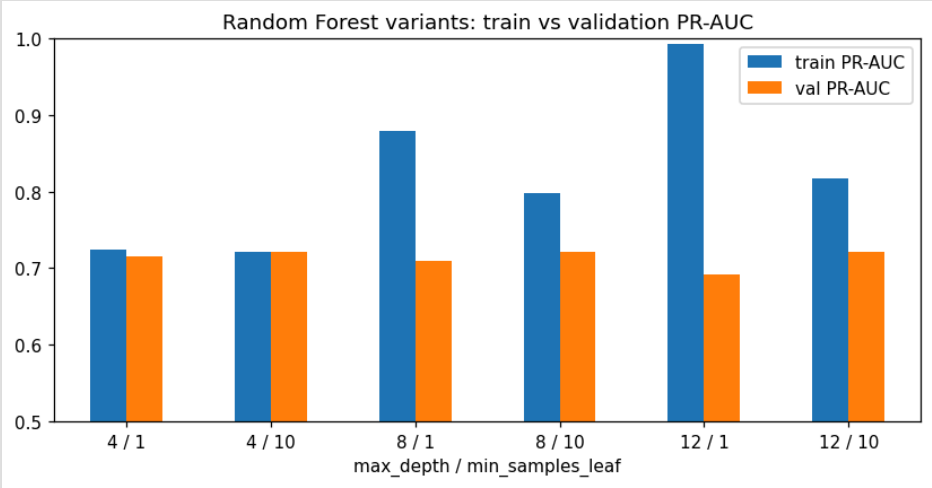


Figure 13. Random Forest hyperparameter variants.

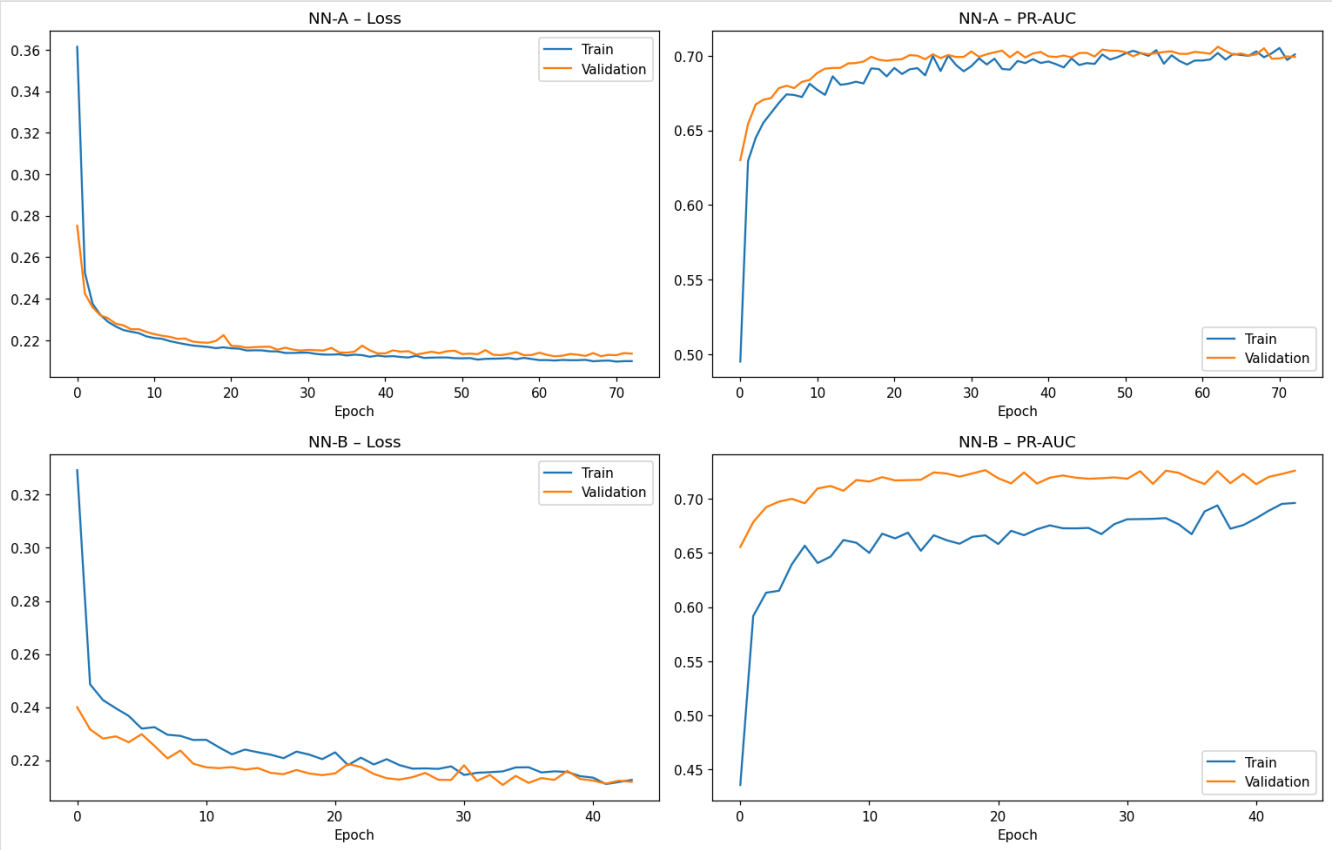


Figure 14. Neural-network training curves.

Metrics. Primary: PR-AUC (average precision). The positive class is rare (~15% podiums) and PR-AUC focuses on how well the real podium finishers are ranked at the top, while ROC-AUC can look high just because there are many easy negatives. **Secondary: F1** at a probability threshold chosen on the validation set (the threshold that maximises validation F1); this is the number that matters when we actually say "podium / no podium". We also report ROC-AUC, Precision and Recall: the spec table lists ROC-AUC, PR-AUC and F1, and we add Precision and Recall as the remaining two of the five metrics because F1 is built from them, so they show why F1 is high or low. Thresholds: LR 0.34, RF 0.39, NN 0.29, all chosen on validation. The test set (2022–2024) is only reported, never used to choose anything.

Table 10. Model comparison, each cell = train / validation / test.

Model	ROC-AUC	PR-AUC (primary)	F1 (secondary)	Precision	Recall
Rule: grid position	0.906 / 0.918 / 0.914	0.616 / 0.668 / 0.647	0.634 / 0.712 / 0.649	0.634 / 0.712 / 0.650	0.633 / 0.712 / 0.647
Logistic Regression	0.925 / 0.931 / 0.935	0.679 / 0.698 / 0.700	0.658 / 0.722 / 0.667	0.605 / 0.646 / 0.610	0.720 / 0.819 / 0.735
Random Forest	0.938 / 0.942 / 0.940	0.722 / 0.722 / 0.708	0.672 / 0.725 / 0.668	0.666 / 0.693 / 0.665	0.679 / 0.761 / 0.672
Shallow NN	0.940 / 0.940 / 0.929	0.712 / 0.728 / 0.689	0.683 / 0.715 / 0.667	0.595 / 0.630 / 0.585	0.802 / 0.827 / 0.775

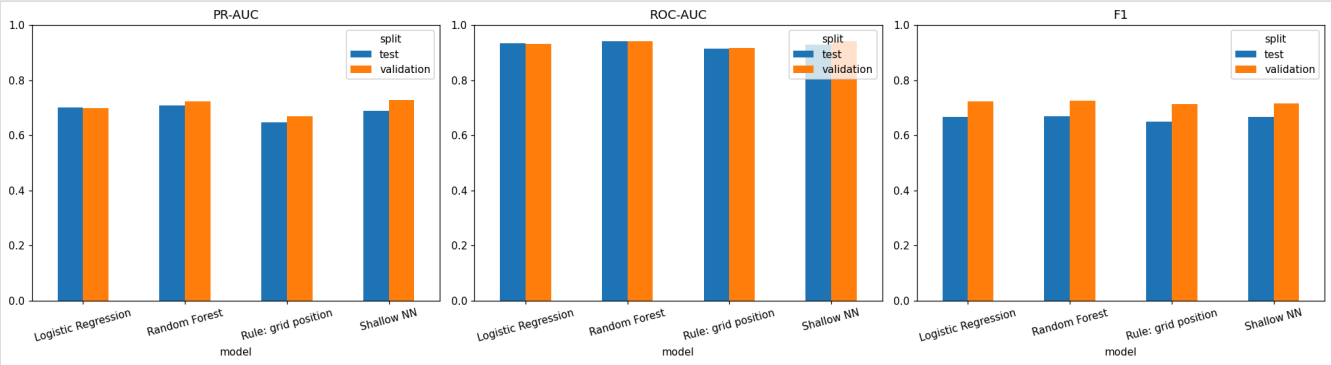


Figure 15. Model comparison on validation and test (PR-AUC, ROC-AUC, F1).

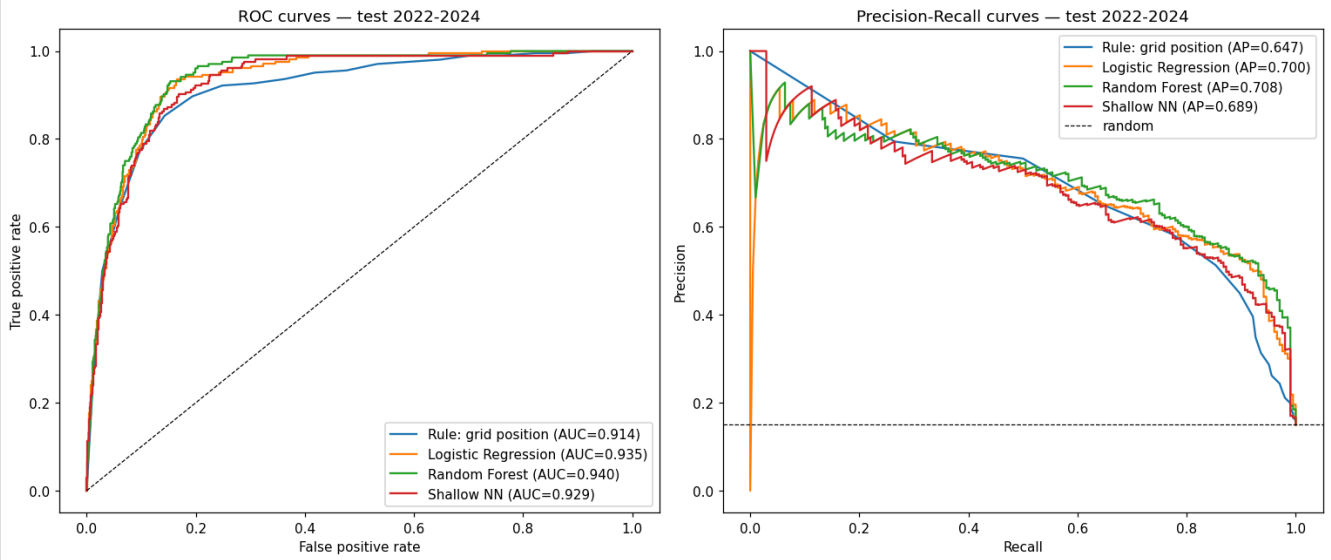


Figure 16. ROC and precision-recall curves of all models on the test seasons 2022–2024.

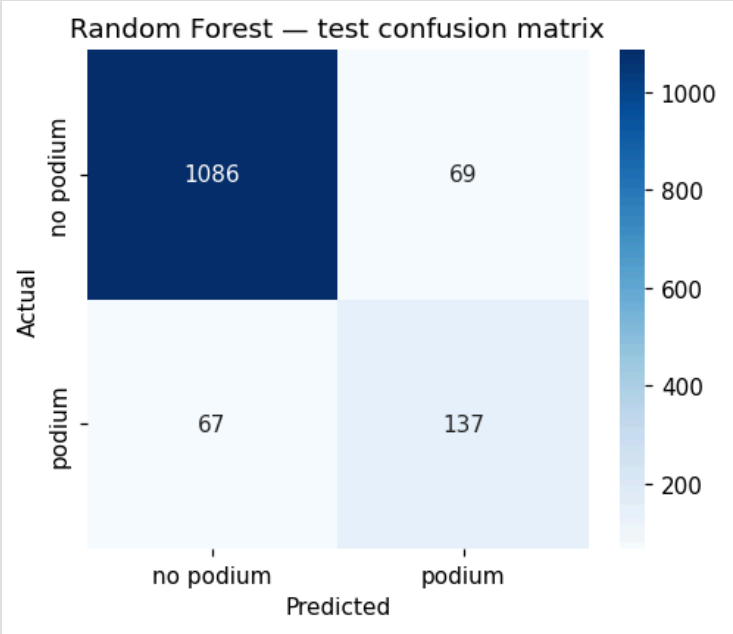


Figure 17. Confusion matrix of the final Random Forest on the test seasons (threshold 0.39).

Interpretation.

- All learned models beat the simple rule: val PR-AUC 0.67 → 0.70–0.73, test 0.65 → 0.69–0.71. The extra features (qualifying pace, form, standings) add information beyond the grid.
- The baseline logistic regression is already strong (val 0.70, test 0.70); the relation "better start + stronger car = podium" is mostly linear.
- **Random Forest** (max_depth 4, min_samples_leaf 10) is the best statistical model: val PR-AUC 0.72, test 0.71, test ROC-AUC 0.94. Deeper trees overfit, so shallow, regularised trees were selected on validation.
- **Shallow NN** scores about the same as the RF on validation (0.728 vs 0.722 PR-AUC) but lower on the test seasons (0.689). Averaged over 5 seeds both reach about 0.718 (see the seed check below). Its validation score is slightly optimistic, because the validation loss was also used to pick the stopping epoch. Its numbers also change a little with the seed and backend (TensorFlow on Kaggle vs PyTorch locally). With ~6k training rows it brings no real gain over trees.
- ROC-AUC is about 0.93–0.94 for every model (most non-podium rows are easy), which is why PR-AUC is the primary metric. At threshold 0.39 the Random Forest finds about 2/3 of the podiums with about 2/3 precision on the unseen test seasons (test F1 about 0.67).
- Train, validation and test are close for LR and RF, so no strong overfitting. The test seasons (new 2022 regulations) are only slightly harder.
- **Final model: Random Forest.** On the primary metric it is tied with the NN: the 5-seed averages are practically identical (our run: 0.718 vs 0.718), and the single-run gap (our run: 0.006) is smaller than the NN's seed-to-seed variation (std about 0.009). It has the best validation F1 and ROC-AUC, is deterministic, needs no scaling, and SHAP can explain it exactly. The choice uses validation results only.

Seed check: how much do the scores move with the random seed?

We call some differences "noise", so we measure it: the selected RF and NN settings are retrained with 5 different random seeds (same data, same features); validation PR-AUC:

Table 10b. Validation PR-AUC over 5 seeds.

model	mean	std	min	max
Random Forest	0.7180	0.0037	0.7132	0.7224
Shallow NN	0.7182	0.0086	0.7052	0.7269

- Averaged over 5 seeds, RF and NN reach practically the same validation PR-AUC (our run: 0.718 vs 0.718).
- The NN varies more between seeds than the RF (our run: std about 0.009 vs about 0.004).
- So a single-run gap between RF and NN says little. Differences of about 0.01 PR-AUC or less (between these models, or between preprocessing orders) are within seed noise.

7. Feature-group ablation

One feature group is removed at a time, Logistic Regression and Random Forest are retrained (same hyperparameters) and the drop in PR-AUC against the full feature set is measured. A large drop means the group adds information the others do not already contain.

Table 11. Feature-group ablation (LR = Logistic Regression, RF = Random Forest).

removed group	model	val PR-AUC	test PR-AUC	test ROC-AUC	val drop	test drop
none (all features)	LR	0.6976	0.7002	0.9349	0.0000	0.0000
none (all features)	RF	0.7219	0.7078	0.9404	0.0000	0.0000
grid	LR	0.6868	0.6923	0.9320	0.0108	0.0079
grid	RF	0.7036	0.6927	0.9355	0.0184	0.0152
qualifying	LR	0.6972	0.7010	0.9357	0.0004	−0.0008
qualifying	RF	0.7116	0.7089	0.9364	0.0103	−0.0010
form	LR	0.7083	0.7006	0.9335	−0.0107	−0.0005
form	RF	0.7248	0.7064	0.9394	−0.0029	0.0015
standings	LR	0.6993	0.6993	0.9339	−0.0016	0.0008
standings	RF	0.7213	0.7006	0.9381	0.0006	0.0072

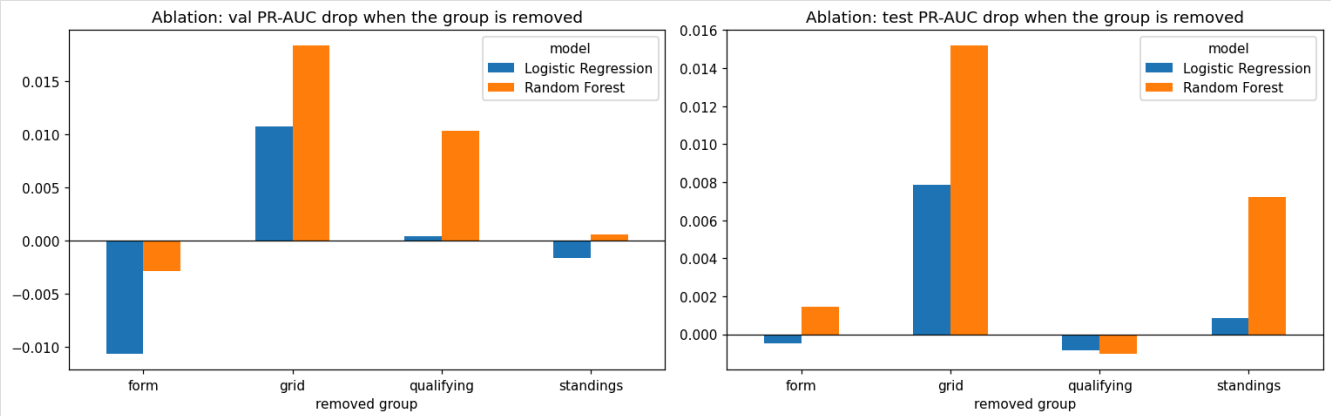


Figure 18. Validation and test PR-AUC drop when each feature group is removed.

- **Grid is the most important group.** Removing it gives the largest drop for both models (RF: −0.018 val / −0.015 test; LR: −0.011 / −0.008).
- **Qualifying:** removing it costs the RF about 0.01 on validation and nothing on test. Most of its information is already in `grid` (correlation 0.9+); only the pace gap is extra.
- **Standings:** a small drop for the RF on test only (−0.007; about 0 on validation). LR is unaffected.
- **Form:** removing it does not hurt (slightly better for LR on validation). Form and standings both measure car/driver strength, so when one is removed the other substitutes.
- **Conclusion:** the information comes mainly from where the car starts plus one measure of car strength. A smaller model would score almost the same. We keep all four groups: the set was chosen **before** training from the EDA evidence, and removing a group never improves the RF on validation **and** test at the same time. Compared with the seed noise measured in Section 6 (RF std about 0.004), the **grid**

drop (0.015–0.018 for the RF) is a real effect, about 4× the noise. The changes of the other groups (at most 0.01) are close to noise level. (The choice is not based on the test numbers.) The ablation also explains why permutation importance gives these groups little credit.

8. Explainability (Random Forest)

We explain the final Random Forest (the tree model lets SHAP compute exact explanations with `TreeExplainer`). **Important: not causal.** All explanations describe what the model relies on to make its predictions, not what causes a podium. A feature being important (e.g. `drv_points_share`) does not mean that having more points makes a driver finish on the podium; it means the model uses it as a signal, often because it is correlated with car and driver strength.

8.1 Permutation importance (test set)

One feature at a time is shuffled and the drop in test PR-AUC is measured (10 repeats). Bigger drop = the model depends more on that feature.

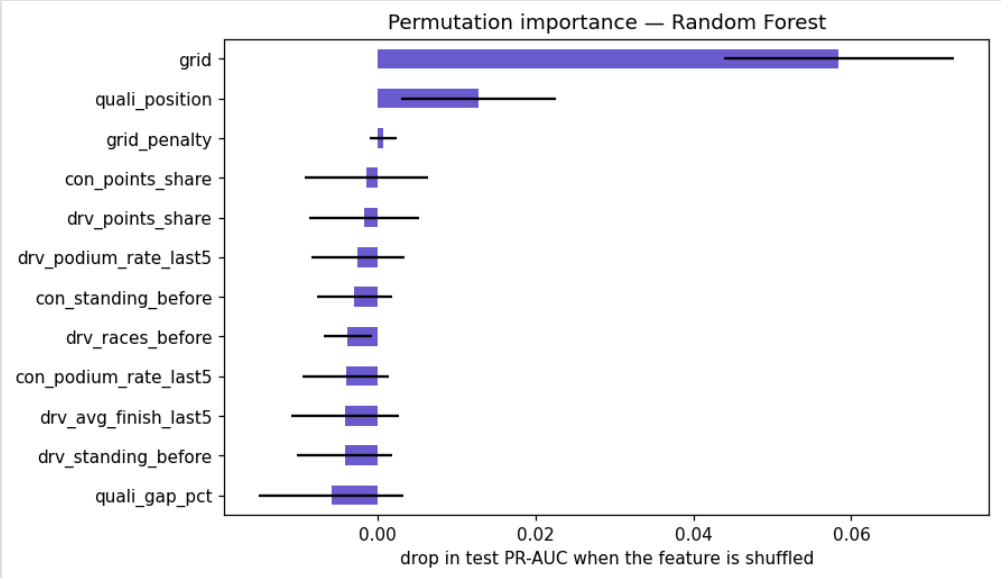


Figure 19. Permutation importance on the test set (drop in PR-AUC).

8.2 SHAP summary and bar plot

SHAP splits each prediction into contributions of the features (positive = pushes towards podium). In the beeswarm each dot is one test row (a sample of 400) and colour is the feature value (red = high).

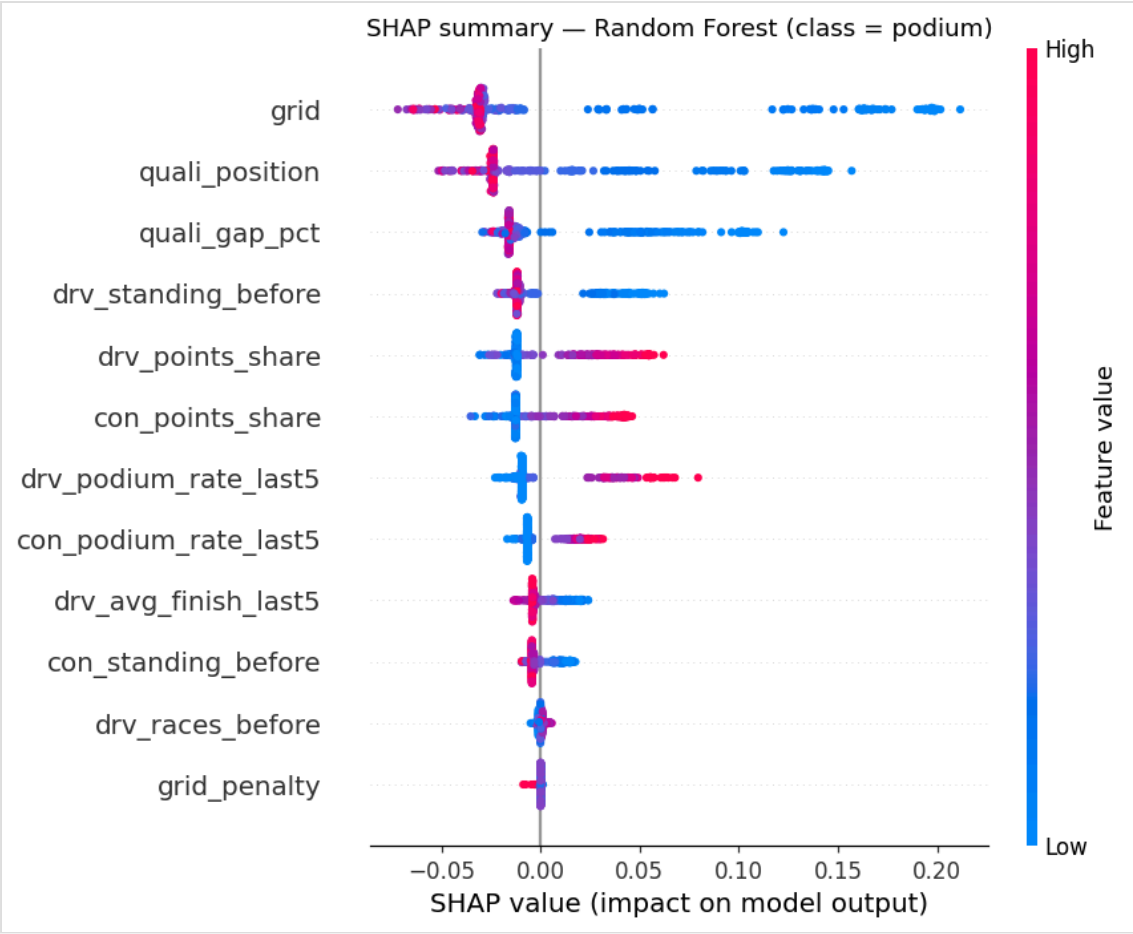


Figure 20. SHAP summary (beeswarm), Random Forest.

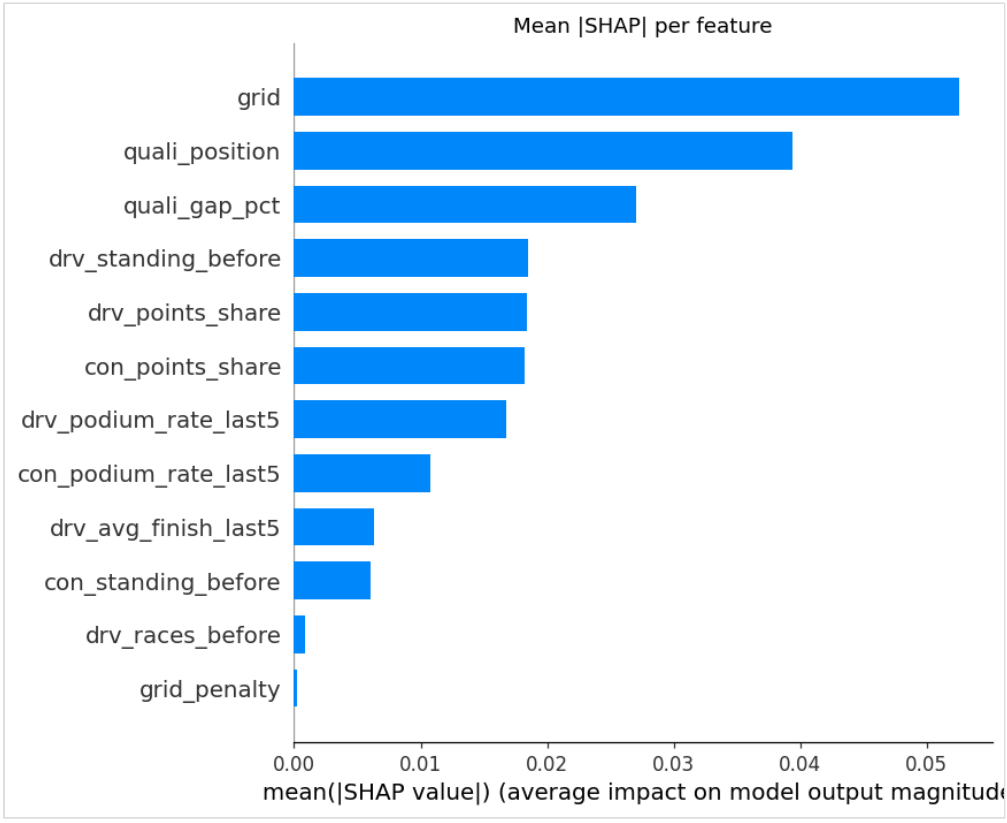


Figure 21. SHAP mean absolute value per feature.

- **Permutation importance:** shuffling `grid` lowers test PR-AUC by about 0.06 (0.0584), `quali_position` by about 0.013, and the other features by about 0 or slightly below 0. A near-zero value does not mean

a feature is useless: when one correlated feature is shuffled the model gets the same information from its "twins" (grid and quali_position, form and standings). This agrees with the ablation.

- **SHAP:** the ranking is grid > quali_position > quali_gap_pct > driver standing / points share > constructor points share > recent podium rate. The direction makes sense: low grid / qualifying positions and small gaps (blue dots) push towards a podium, by up to +0.2 probability. High points shares and recent podium rates (red dots) push it up. Starting further back pushes down only a little (about −0.05), because most rows are already near the base rate of 14%.
- SHAP shares credit among correlated features, while permutation importance only credits features that cannot be replaced. That is why SHAP shows form and standings as useful and permutation importance does not.
- **Again, these are associations the model uses, not causes.** A high points share does not cause a podium; it is a signal of a fast car/driver, and that is what produces both.

8.3 One prediction explained: SHAP waterfall and LIME

Test row: **Lando Norris, 2024 Italian GP (Monza)**. He started from pole (P1) and finished 3rd (a podium). The model's podium probability is 0.758. Feature values: grid 1, grid_penalty 0, quali_position 1, quali_gap_pct 0, drv_podium_rate_last5 0.6, drv_avg_finish_last5 6.2, con_podium_rate_last5 1.0, drv_races_before 119, drv_standing_before 2, drv_points_share 0.763, con_standing_before 2, con_points_share 0.931.

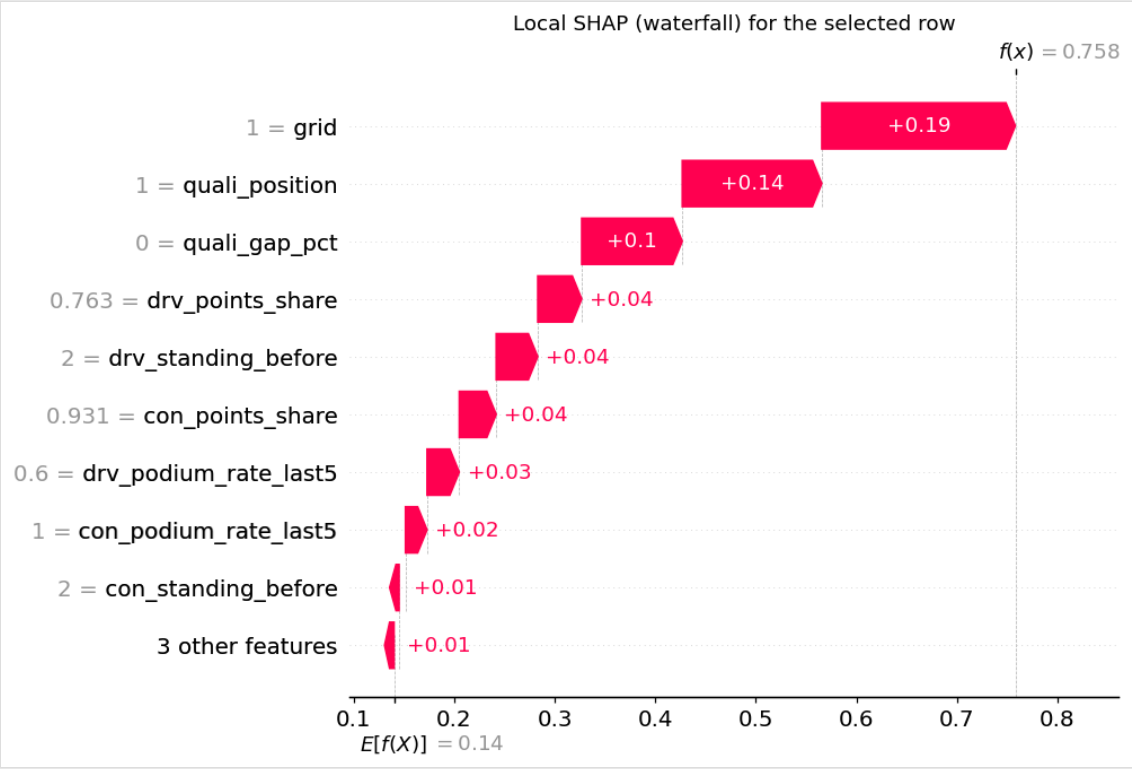


Figure 22. SHAP waterfall for Norris, Monza 2024.

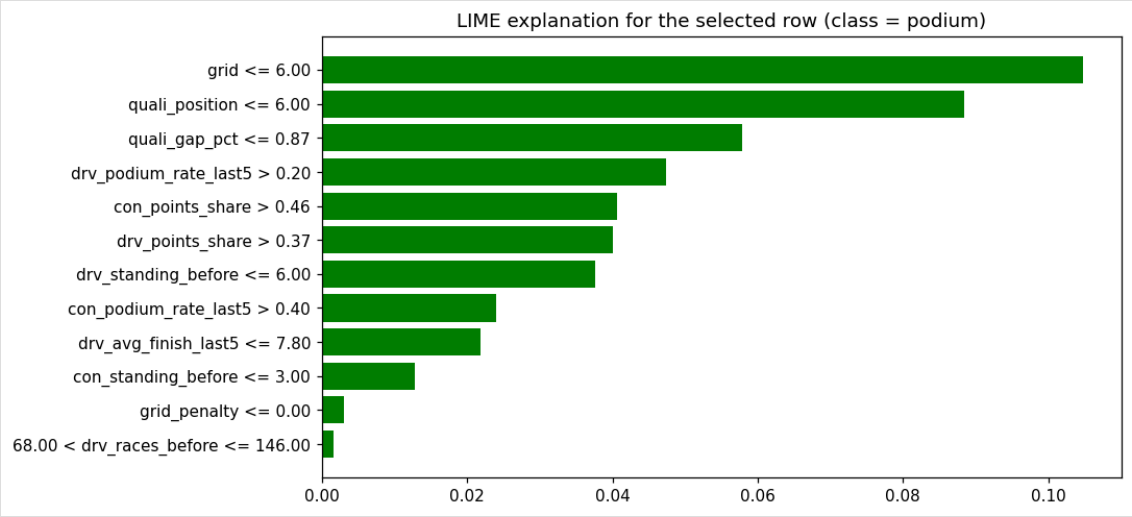


Figure 23. LIME explanation for the same row.

- The model gives 0.76 probability, above the 0.39 threshold, so it predicts podium. This is correct.
- **SHAP waterfall:** starting from the average prediction (0.14), the largest pushes up come from grid = 1 (+0.19), quali_position = 1 (+0.14) and quali_gap_pct = 0 (+0.10), i.e. he was on pole. Smaller pushes come from his strong season: points share 0.76 and P2 in the championship (+0.04 each), team points share 0.93 (+0.04), and podiums in 3 of his last 5 races (+0.03).
- **LIME** shows the same story with rules in probability units: `grid <= 6` (0.105), `quali_position <= 6` (0.088) and `quali_gap_pct <= 0.87` (0.058) have the largest positive weights, followed by the form and standings rules. SHAP and LIME agree on the order of the top features; their numbers differ because LIME fits a local linear model on perturbed samples while SHAP splits the exact prediction.
- **Not causal:** the explanation says why the model gave this probability, not why Norris finished on the podium.

9. Inference function

`predict_podium(record)` takes a dictionary with the pre-race features of one driver in one race and returns the podium probability, the yes/no decision (using the threshold 0.39 chosen on validation) and the list of features that were missing. Missing features are allowed: if the qualifying position is missing but the grid slot is known, it is set to the grid slot (and `grid_penalty` to 0), because the median (P11) would contradict a known grid slot (e.g. grid 1 with qualifying 11). All other missing features are filled by the same median imputer fitted on the training data (inside the model pipeline). So the function still works, e.g., for a rookie with no form history or a race without qualifying data.

Table 12. Inference demos.

Demo	Input	Output
1. Complete record	all 12 pre-race features of the Norris/Monza row	probability 0.758, predicted podium = True, missing = none
2. Partially missing	same row without `grid_penalty`, `quali_position`, `quali_gap_pct`, `drv_standing_before`	probability 0.609, predicted podium = True, missing (imputed) = those 4 features
3. Very sparse	only `grid` (P4) is known (e.g. a debutant with no history)	probability 0.123, predicted podium = False, missing (imputed) = the other 10 features

What the demos show.

- **Complete record (pole, strong season):** high podium probability (about 0.76), predicted podium. This is correct, as he finished 3rd.

- **Partial record:** the qualifying position is missing, so it is set to the grid slot (P1). The qualifying gap and the standing are filled with the training medians (a midfield gap of about 1.8%). The model loses the information "he was the fastest by a margin", so the probability drops (0.76 to about 0.61), but it still stays above the 0.39 threshold thanks to grid = 1 and the strong form/points. Missing information makes the model more cautious; it does not crash.
- **Sparse record:** with only grid P4 and no history, every other feature takes a median (weak) value, so the probability is low (about 0.12). The function always returns the list of imputed features, so the user knows how much was guessed.

10. Conclusions & known limitations

Summary

- **Data:** the 14 raw tables were audited (all primary keys unique, all foreign keys reconcile). 160k `\N` sentinels became real missing values, dates and durations were typed, two label inconsistencies fixed, and 91 duplicate driver-race rows (1950s shared drives) were removed with a documented rule. The modeling table has one row per `(raceId, driverId)`: 8,970 rows, 2003–2024.
- **Q1:** front-row conversion is highest at Monza/Monaco before 2014 and at Suzuka/Yas Marina/Monaco after; the ranking changes strongly (Spearman about 0.21) and grid is not the same as qualifying because of penalties. **Q2:** no home advantage once grid position is controlled. **Q3:** Caterham/Lotus/Toro Rosso/McLaren (2014–2021) and Alfa Romeo/Alpine (2022–2024) had the highest mechanical-retirement rates; reliability improved overall.
- **Model:** 12 leakage-safe features in 4 groups. The Random Forest reaches PR-AUC about 0.71, ROC-AUC about 0.94 and F1 about 0.67 on the unseen 2022–2024 seasons, beating the logistic-regression baseline and the grid-only rule. The shallow NN is comparable on validation but slightly worse on test.
- **Main drivers of the prediction:** grid and qualifying, then car/driver strength (standings and form). These are model associations, not causal effects.

Known limitations

- **Qualifying coverage drift:** qualifying data is missing for most races before 2003, so the model is trained only on 2003–2024. It is not valid for earlier eras without qualifying data; the inference function then relies on imputation, which pulls predictions towards the average.
- **Era sensitivity:** rule changes (hybrid engines in 2014, new cars in 2022) change how dominant teams are, and the test seasons are a new regulation era. Points systems changed (we use points shares to reduce this). Field size and race count per season also vary.
- **Correlated features:** grid, qualifying, form and standings overlap, so single-feature importance values must be read together with the ablation.
- **Simplifications:** pit-lane starters are placed at the back of the grid; best qualifying time is the minimum of Q1/Q2/Q3 (session conditions can differ); home country is taken from nationality, not birthplace; the list of "mechanical" statuses is a judgement call; sprint results are not used.
- **Calibration:** the model outputs are used as scores for ranking and the threshold. They are not calibrated probabilities (no reliability curve / Brier score was checked), so "0.76" should be read as "very likely relative to other entries".
- **Qualifying format:** in 2003–2005 qualifying used single laps with race fuel / aggregate times, so `quali_gap_pct` in those seasons mixes pace and fuel load.
- **Each row is predicted independently:** the model is not forced to predict exactly 3 podiums per race.
- **Threshold:** 0.39 maximises F1 on 2018–2021. A different use (e.g. betting, where precision matters more) would need a different threshold.